

Day 55 -- Line-by-Line Syntax Explanation

day55_data_pipeline_efficiency.py, segment by segment | MD Mutasim Billah

1. write_dataset_to_disk() -- Simulating a Real Image Folder

one file per image, not one file for the whole array

```
for i in range(len(y)): # loop over every image
    path = os.path.join(root, f"img_{i:05d}.npy") # this image's OWN file
    np.save(path, X[i]) # write just ONE image to its own file
    paths.append(path) # remember this path for the loaders below
```

This deliberately mimics how real image datasets are stored (e.g. one .jpg per file in an ImageFolder layout), not this series' usual single big array. The whole point of today's lesson only makes sense against data that is genuinely split across many files.

2. InMemoryLoader vs. LazyDiskLoader -- Where the Disk Read Happens

InMemoryLoader: every file read ONCE, in __init__

```
def __init__(self, file_paths, y, batch_size, seed):
    self.X = np.stack([np.load(p) for p in file_paths]) # read EVERY file, right now
    ...
    def epoch_batches(self):
        ...
        yield self.X[idx], self.y[idx], idx # pure slicing -- no disk access here
```

All disk I/O for this loader happens exactly once, at construction time. Every subsequent batch access, across every epoch, is pure in-memory array slicing -- the fastest possible access pattern, at the cost of needing the whole dataset's memory footprint up front.

LazyDiskLoader: every file read EVERY time it's needed

```
def _read_batch(self, idx):
    imgs = [np.load(self.file_paths[i]) for i in idx] # disk read, EVERY call, no caching
    return np.stack(imgs), self.y[idx], idx
def epoch_batches(self):
    ...
    yield self._read_batch(idx) # reads from disk RIGHT NOW, synchronously
```

The constructor does no I/O at all -- just stores paths. Every single batch, in every epoch, re-reads its images from disk from scratch. This is the real, measured source of LazyDiskLoader's 25% slowdown versus InMemoryLoader in this lesson's benchmark.

3. PrefetchDiskLoader -- Overlapping I/O With Compute

reading the NEXT batch while the caller uses the CURRENT one

```
future = self.executor.submit(self._read_batch, batch_idx[0]) # start reading batch 0 NOW
for i in range(len(batch_idx)):
    Xb, yb, idx = future.result() # wait for the batch THIS iteration needs
    if i + 1 < len(batch_idx):
        future = self.executor.submit(self._read_batch, batch_idx[i + 1]) # start the NEXT one
    yield Xb, yb, idx # hand this batch over -- next one already loading
```

This is the whole trick: by the time the caller finishes using batch *i* (e.g. a training step) and asks for batch *i*+1, a background thread has often ALREADY finished reading it -- so `future.result()` returns almost immediately instead of blocking on a fresh disk read. This is a from-scratch, one-thread version of exactly what `num_workers>0` does in a real `DataLoader`.

4. `demo_batch_equivalence_check()` -- Verifying Before Trusting

comparing all three loaders' output, batch by batch, in lockstep

```
for (Xb_m, yb_m, idx_m), (Xb_l, yb_l, idx_l), (Xb_p, yb_p, idx_p) in zip(
    mem.epoch_batches(), lazy.epoch_batches(), pre.epoch_batches()):
    idx_ok = np.array_equal(idx_m, idx_l) and np.array_equal(idx_m, idx_p) # same shuffle order
    img_ok = np.array_equal(Xb_m, Xb_l) and np.array_equal(Xb_m, Xb_p) # same pixels, bit-for-bit
    lab_ok = np.array_equal(yb_m, yb_l) and np.array_equal(yb_m, yb_p) # same labels
```

`zip()` walks all three generators in lockstep, one batch position at a time, so each loader's *N*th batch is compared directly against the other two's *N*th batch. `np.array_equal` checks EXACT equality (not `np.allclose`) since these should be bit-for-bit identical -- the same image data, just fetched through three different code paths.

5. `TinyConvNet.flat_weights()` -- an Exact-Equality Check, Not Just "Close"

concatenating every parameter into one vector for a single comparison

```
def flat_weights(self):
    return np.concatenate([self.Wc.ravel(), self.bc.ravel(),
        self.W1.ravel(), self.b1.ravel(),
        self.W2.ravel(), self.b2.ravel()])
```

Flattening every weight/bias array into one long vector lets `demo_training_equivalence()` compute a single max-absolute-difference number across the ENTIRE network in one line, rather than comparing six separate arrays by hand. The result (0.00e+00) means the two networks are not just close, but bit-for-bit identical after 30 epochs -- exactly what deterministic floating-point math on identical inputs should produce.